

Сессионные Tinder-like рекомендации

Постановка задачи

Исходная идея

- Сессионная рекомендательная система
- Sequential-модель SASRec
- Задача next-item prediction



Новый подход

- Датасет: пары юзер-айтем + фичи
- Чистый ранкер-датасет
- Задача: динамический реранкинг
- Адаптация под сессионный интерес

Идея проекта

Обертка поверх «базовой» рекомендательной системы Авито

1

Tinder-like интерфейс

Пользователь лайкает / скипает
карточки — явный сигнал о текущем
интересе

2

Динамический реранкинг

После первого лайка оставшиеся
объявления переранжируются

3

Сессионный профиль

Каждое новое взаимодействие
уточняет профиль и обновляет
порядок выдачи

Ключевой вопрос: можно ли несколькими лайками поднять релевантные объявления выше в выдаче?

Как работает система



Зачем это нужно

1

Cold start и онбординг

Tinder-like интерфейс позволяет быстро собрать сильные сигналы о новом пользователе и сразу адаптировать рекомендации без долгой истории поведения.

2

Уточнение текущего интереса

Профиль меняется с контекстом. Пользователь обычно смотрит авто, но сейчас ищет детскую коляску — несколько лайков мгновенно корректируют выдачу.

3

Выход из filter bubble

Выдача формируется как комбинация: базовое ранжирование + сессионный сигнал + exploration-компонент для сохранения разнообразия рекомендаций.

Оффлайн-оценка

Для каждой сессии: лайк на позиции $k \rightarrow$ момент реранкинга. Берём кандидатов $k+1..N$ и переставляем. Цель — поднять последующие лайки выше в выдаче.

NDCG@10

Основная метрика. Насколько хорошо реранкер выводит релевантные объявления в топ-10 результатов

Mean Rank Lift

На сколько позиций в среднем поднялось следующее лайкнутое объявление после реранкинга

Pct Lifted

Доля сессий, где хотя бы одно лайкнутое объявление поднялось выше исходной позиции

Результаты экспериментов

Метод	NDCG@10	Rank Lift	Pct Lifted
Исходный порядок	0.1985	—	—
Cosine Similarity	0.3036	+5.8	71%
CatBoost	0.3444	+7.1	78%

x1.73

рост NDCG@10
0.1985 → 0.3444

78%

случаев, когда
релевантный объект
поднялся

Демо-приложение

FastAPI

REST API: запросы ленты,
регистрация лайков и скипов

Docker Compose

Все сервисы упакованы
в контейнеры и оркестрируются

Redis + TTL

Сессионное хранилище:
событие {item_id, тип}

ЕМА-перанкинг

```
scores = (1 - w) x scores + w x new_scores
```

```
w = 0.70
```

```
new_scores = CatBoost или cosine similarity (fallback)
```

Эмбединги и ANN-поиск

Эмбединги: SigLIP

Для каждого айтема:

Item_embedding = лин. комб. (title_embedding, image_embedding)

Запросный вектор сессии

```
query = mean(liked) - a x mean(skipped)
query /= norm(query)      (a = 1.0)
```

Скипы отталкивают запрос от нежелательных направлений

FAISS: расширение кандидатов

- Проблема: ограниченность исходной выдачи
- FAISS — дополнительный источник кандидатов
- ANN-поиск по усредненному эмбедингу последних k лайков
- Результаты подмешиваются в выдачу

query -> FAISS -> top-K -> выдача

Итоги

x1.73

рост NDCG@10
0.1985 → 0.3444

78%

случаев с подъёмом
релевантного объекта

3

компонента выдачи:
base + сессия + exploration

Несколько явных взаимодействий пользователя достаточно для существенной адаптации выдачи